

Sound Synthesis

04



Have you watched the movie Jurassic Park?

Did you know that the sounds of dinosaurs in that movie were created by mixing noises made by animals such as tortoises and geese? Let's see if we can create some realistic sounds using this technique.



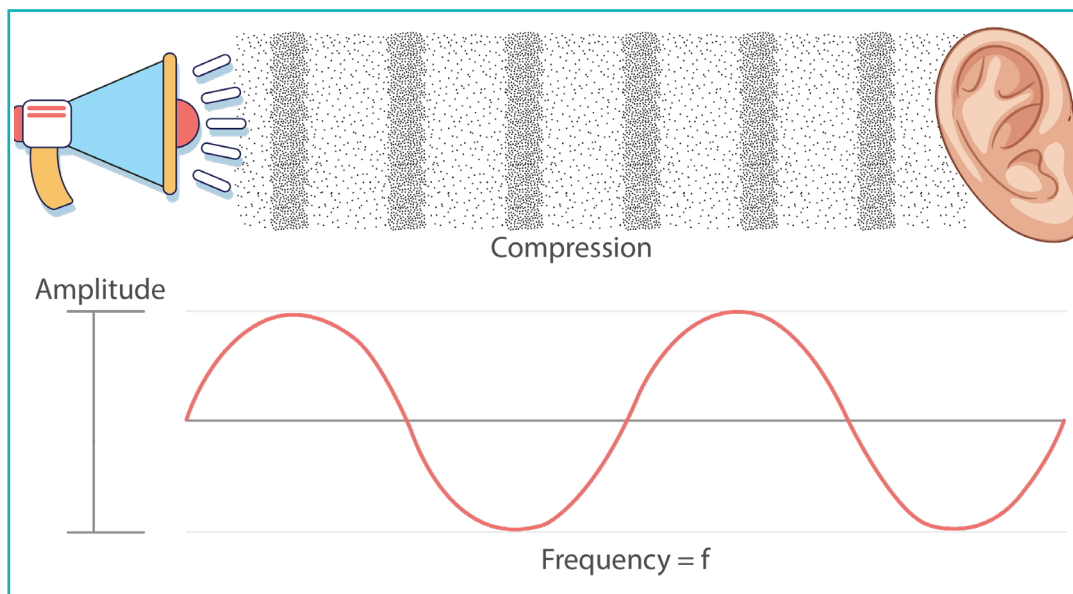
The digital sound production has enabled the recreation of various complex sounds like sounds in space or sounds of extinct animals. Let us discover more about such sounds and their creation.

'Sound' is a form of energy that is created by vibrations. These vibrations travel through molecules of air, water, or solid to reach our eardrums, which also vibrate and transmit a signal to the auditory nerve in the brain. This is known as the 'auditory process'.

Sounds in nature, human voices, music etc. are made by vibrations generated because of pressure waves. We can also digitally generate various artificial sounds using technology. We can simulate sounds of nature, including those of extinct species of animals such as dinosaurs.



We know that sound travels in the form of waves. A 'sinusoidal wave' is the most fundamental representation of a sound wave. The 'frequency' of each wave represents the number of repetitions or oscillations per unit time, i.e., how fast a single cycle of oscillation is repeated. The 'amplitude' of a wave determines the loudness of the sound, i.e., the volume of the audio.



4. Sound Synthesis

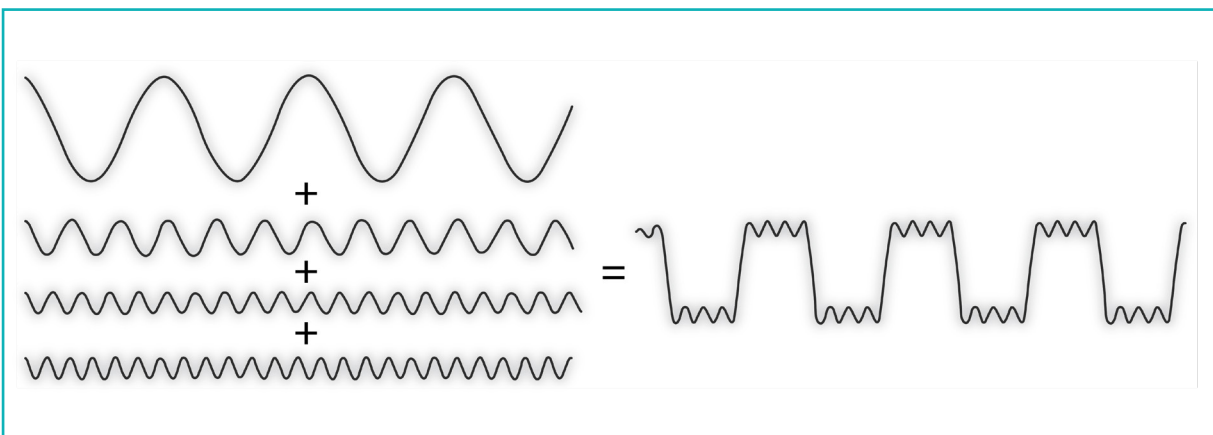
Bytes

Only waves between frequency of **20 Hz - 20 kHz** are audible to the human ear. For example, open this link to find out what a sound wave for frequency 200 Hz sounds like. <https://www.szynalski.com/tone-generator/>

A QR code that takes you to the link.



Sine waves of different frequencies have varied pitches of sound. The sounds around us are usually a very complex combination of multiple sine waves of different frequencies and amplitudes.



This combination generates the complex sounds that we hear. A graph depicting speech, song, or any other ambient sound looks similar to the given image.



Sound Synthesis

Mathematics can help figure out patterns of the combinations of various sounds. Complex natural sounds can be recreated using those patterns. This technique, used to digitally recreate sounds from scratch using electronic hardware or software, is known as **'audio synthesis or sound synthesis'**.



Applications of Sound Synthesis

- The most common application is the synthesis of musical instruments on digital devices like synthesizers.
- Create distinctive and unique digital sounds that cannot be created physically. Example: Digital music and sounds of electronic musical instruments.
- Recreating the sounds of acoustic instruments enables the production of their musical sounds digitally.
- Recreate natural sounds as a background score for media content.
- Create automatic systems and processes for generating speech from text using the software.



Real-Time Audio Synthesis

Games and media installations require sound to be played in real-time, in synch with the changes in the surroundings, based on user inputs or visuals displayed on the screen. Sound should be generated based on movement, player position, or the user in media installation. This technique of generating sound in real-time is known as a '**real-time sound/audio synthesis**'.

For example, the sound of water ripples when a person's hand is moving in the water can be synthesized.



Exercise

Fill in the Blanks.

- 1 The number of oscillations per unit time of a wave is _____.
- 2 The _____ of a sound wave represents the volume of the sound.

Activity

In this lesson, we will create a real-time sound synthesis of water waves simulation using JavaScript in p5js. This can be created using a short audio file of flowing water as a building block. Real-time processing will be added to modify both the amplitude and frequency.

Step 1: Open JS Code

- Open 'p5js editor' and rename the project as "sound_synthesis".
- Download the 'water.wav' audio file using the QR code.

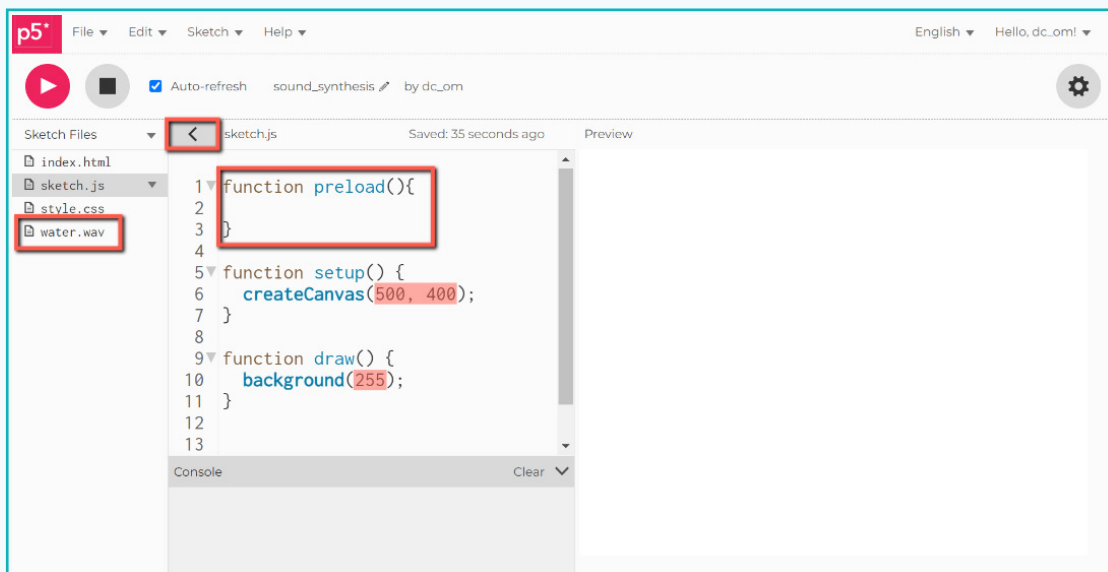


Step 2: Load the Audio File

Note that a 'p5.sound.min.js' JavaScript file is linked to the 'index.html' file in project files. It allows us to use commands from the 'p5js sound library'.

Now we can start with the activity by uploading the base audio file that we will use for audio synthesis of water sounds.

- 1 Upload the 'water.wav' file to the sketch files of the project.
- 2 Add the 'preload' function to load the audio file in the code.
- 3 Change the dimensions of the canvas to '500 x 400' pixels and set the background colour to 'white'.



Note:

In p5js, the 'sound library' provides commands and functions related to sound. They can be explored by searching for the p5js sound in the reference folder or by using the following link:

<https://p5js.org/reference/#/libraries/p5.sound>

Two important commands for our activity are 'play' and 'loop'.

.play() command is used to play the audio only once.

.loop() command is used to play the audio in a loop.

Activity

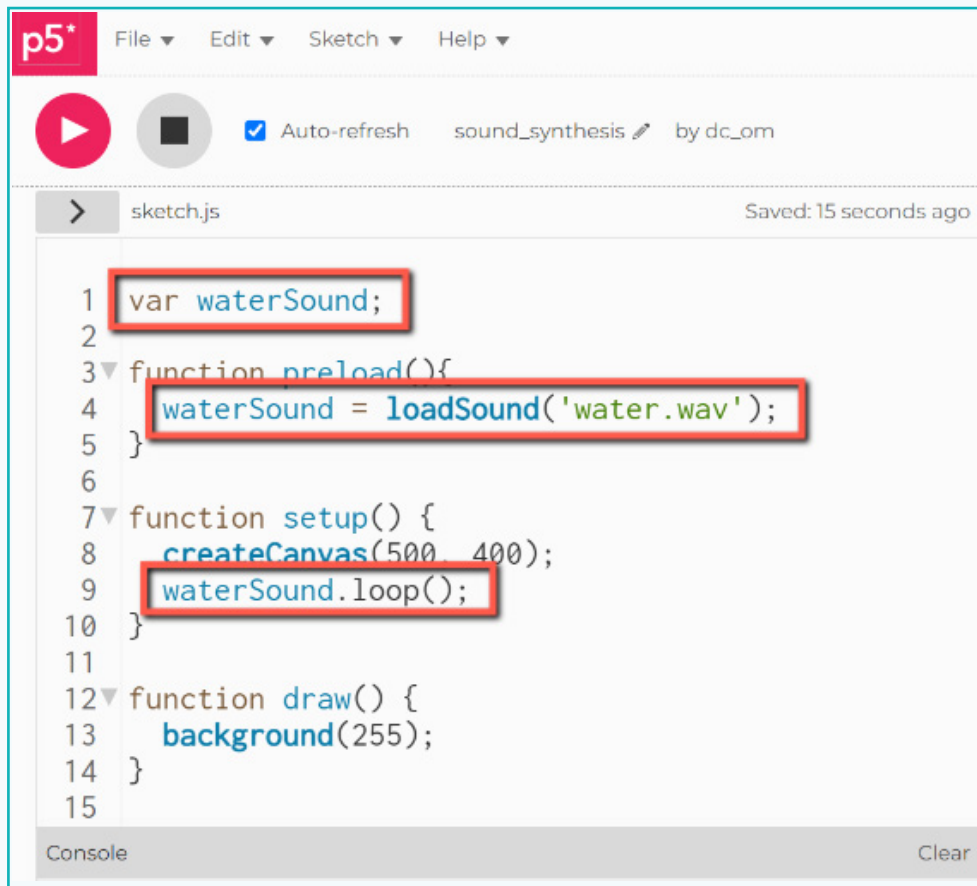
Step 3: Add Audio Loop

We will use a loop as we want to continuously play the audio. We will later control the volume and frequency based on real-time input.

- 1 Create a variable 'waterSound' to save and load the audio file in the code at the beginning of the program.
- 2 Use the 'loadSound' command to load the audio in the 'waterSound' variable in preload function.

Activity

- 3 Add the 'waterSound.loop()' command in the setup to continuously play the audio file in a loop.



```
1 var waterSound;
2
3 function preload(){
4   waterSound = loadSound('water.wav');
5 }
6
7 function setup() {
8   createCanvas(500, 400);
9   waterSound.loop();
10 }
11
12 function draw() {
13   background(255);
14 }
15
```

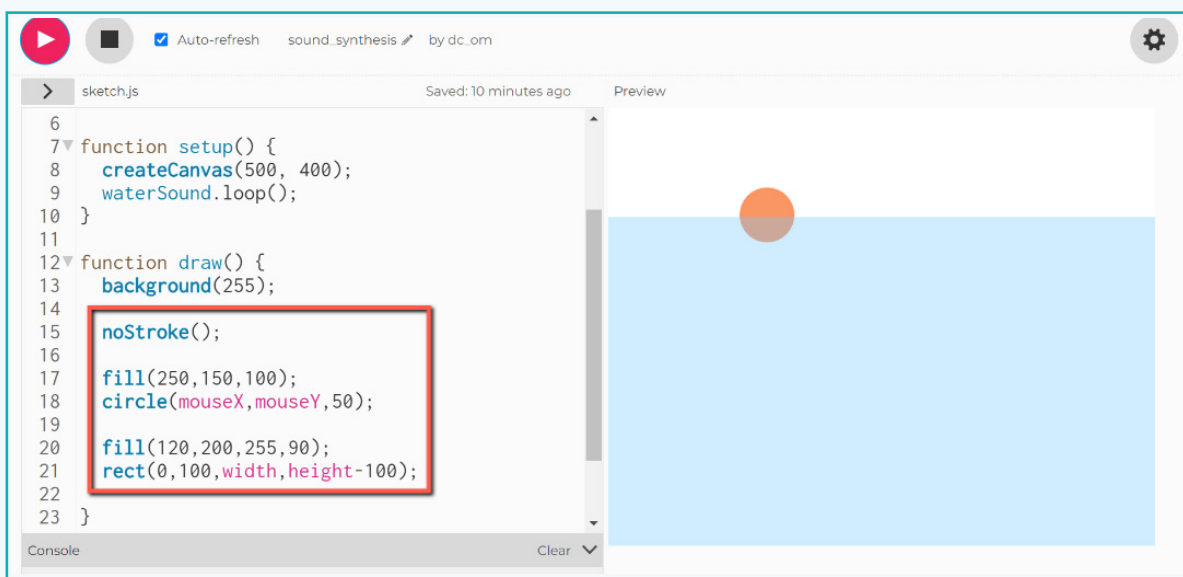
Now, we will be able to hear water flowing. We can now create visuals of the water and an object that can be moved into the water using the mouse pointer.

Step 4: Add Visuals

- 1 Add the 'noStroke()' command to draw shapes without borders.
- 2 Draw a circle of '50 pixels' in diameter at the mouse cursor position and fill it with any colour to represent the object in the water.

Activity

- 3 Draw a rectangle to show water with colour values: 'R=120', 'G=200', 'B=255', and 'A=90' (transparency) on the screen. Water should fill the entire canvas except '100 pixels' at the top of the screen.



In this simulation, we will replicate the sound that would be heard when the listener is positioned at the object's location. In this setup, we will be dealing with different sounds of water based on its location.

Case 1: Object Out of Water

- When the object has fully emerged out of the water, there will be a gentle background sound of a water pool.

Case 2: Object Inside Water

- When an object in water moves, water moves with it, creating waves. The wave sounds will be generated based on how fast the object moves in the water.

Activity

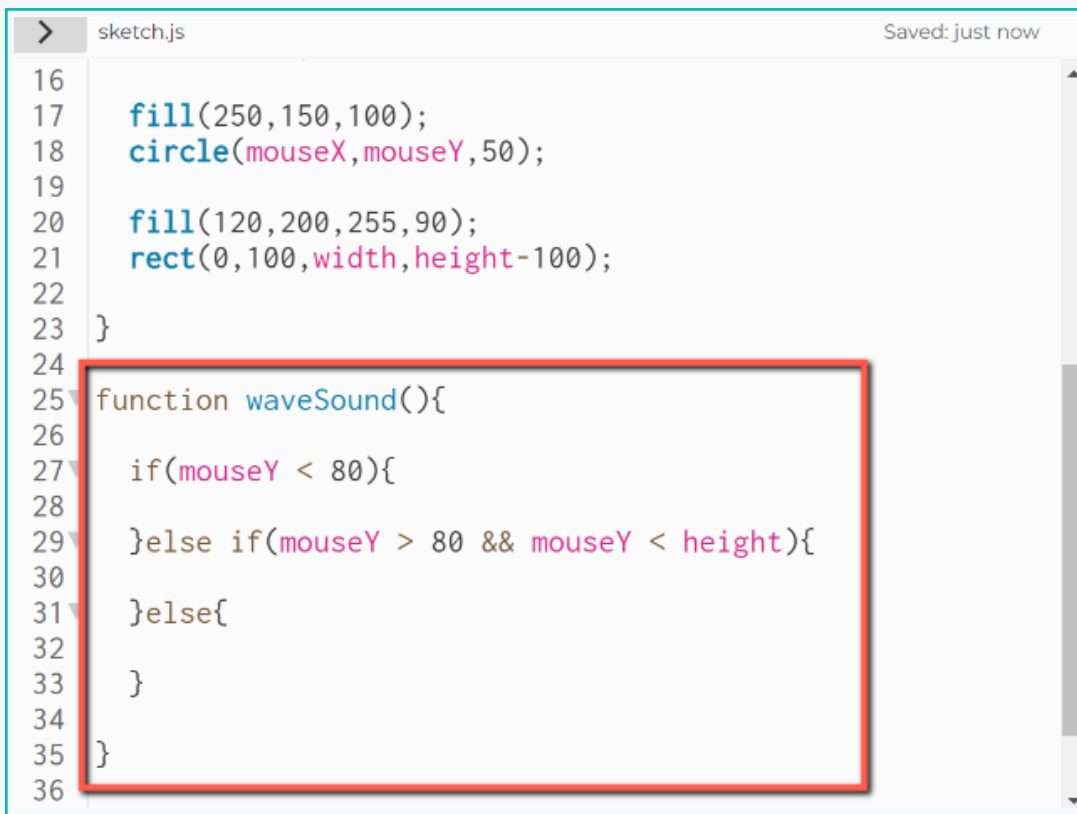
With this distinction, we can create 2 scenarios based on the y position of the mouse or object:

- a. Outside Water - **mouseY < 80**
- b. Inside Water - **mouseY > 80 && mouseY < height**

We can check this using 'console.log(mouseY)'.

Step 5: Simulate the Sound of Water

- a. Create a user-defined function named "waveSound()".
- b. Add an **if ()... else if()... else() condition** to implement scenarios for different sound effects, as discussed.



```
> sketch.js Saved: just now
16
17 fill(250,150,100);
18 circle(mouseX,mouseY,50);
19
20 fill(120,200,255,90);
21 rect(0,100,width,height-100);
22
23 }
24
25 function waveSound(){
26
27   if(mouseY < 80){
28
29   }else if(mouseY > 80 && mouseY < height){
30
31   }else{
32
33   }
34
35 }
36
```

Activity

Step 6: Change the Volume of the Sound

- Create a local variable 'vol' in the function and initialize the variable value to '0'.
- If the first condition is 'true', set the 'vol' to '0.02' for ambient water sound. If none of the conditions is met, i.e., in the 'else' section, set the 'vol' value to '0'.



```
> sketch.js Saved: 2 minutes ago
17 fill(250,150,100);
18 circle(mouseX,mouseY,50);
19
20 fill(120,200,255,90);
21 rect(0,100,width,height-100);
22
23 }
24
25 function waveSound(){
26   let vol=0;
27
28   if(mouseY < 80){
29     vol = 0.02;
30   }else if(mouseY > 80 && mouseY < height){
31
32   }elseif
33     vol = 0;
34 }
```

In the case of the object being underwater, i.e., if the second condition is true, the sound of the water depends on the movement of the object. If the object is moved horizontally, it will create waves in the water. Hence, we will have to generate the sound of waves depending on how fast the object moves.

We can calculate the speed of the movement of the object using a default variable 'movedX'. It represents the value of the movement of the mouse cursor in x direction per unit time. We can correlate the speed of movement to the volume of the sound. We can set up the volume of the sound being played using the 'amp()' command.

.amp(amplitude): It sets the amplitude of the sound and accepts input values between the values of '0' and '1'. The input value '1' corresponds to the full volume of that sound file.

Note that the 'movedX' value when the mouse is moved to the left is negative. We can verify by using the 'console.log()' command. The sound generated by the waves depends only on the magnitude. It does not depend on the direction. Therefore, we can use the 'abs()' method to get only the positive(+ve) values of movedX i.e. magnitude.

Activity

Step 7: Add Variables

- Create a local variable 'xSpeed' and assign a '+ve' magnitude value of the default variable 'movedX' using the 'abs()' command.
- Map the value in the variable 'xSpeed' from range '0-10' (mouse velocity in the x direction) to '0.01 - 1' (volume of the sound).

```
> sketch.js Saved: just now
17 fill(250,150,100);
18 circle(mouseX,mouseY,50);
19
20 fill(120,200,255,90);
21 rect(0,100,width,height-100);
22 }
23
24 function waveSound(){
25   let vol=0;
26
27   if(mouseY < 80){
28     vol = 0.02;
29   }else if(mouseY > 80 && mouseY < height){
30     let xSpeed = abs(movedX);
31     vol = map(xSpeed,0,10,0.01,1);
32   }else{
33     vol = 0;
```

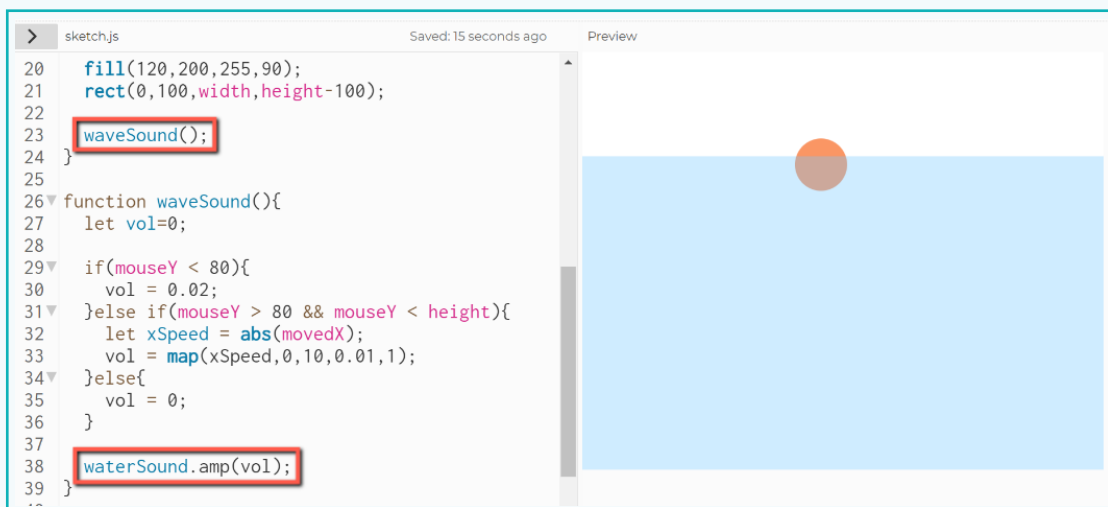
Note:

The lower range of the volume is '0.01' as there should be some ambient sound even if there is no object movement. Once we have all the values for the amplitude, we can use the 'amp()' command to set the volume.

Activity

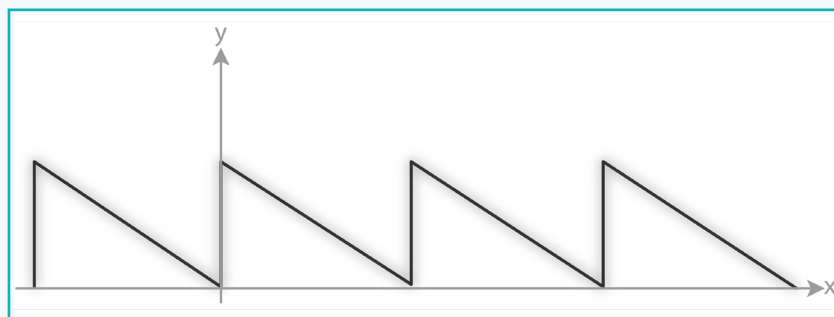
Step 8: Set the Volume

- Set the value in the 'vol' variable as the amplitude of the sound using the 'waterSound.amp(vol)' command outside of the condition.
- Call the function 'waveSound()' in draw and test the code.



```
20 fill(120,200,255,90);
21 rect(0,100,width,height-100);
22
23 waveSound();
24
25
26 function waveSound(){
27   let vol=0;
28
29   if(mouseY < 80){
30     vol = 0.02;
31   }else if(mouseY > 80 && mouseY < height){
32     let xSpeed = abs(movedX);
33     vol = map(xSpeed,0,10,0.01,1);
34   }else{
35     vol = 0;
36   }
37
38   waterSound.amp(vol);
39 }
```

We hear an intermittent abrupt sound of water when the mouse is moved in the water. But in the case of waves, the volume of the sound increases rapidly but dies down slowly in a downward ramp. We can approximately depict the graph of the volume using a ramp function.



This can be achieved when the amplitude of the sound reduces slowly like a ramp. The 'amp()' command takes another optional input parameter known as the 'rampTime'.

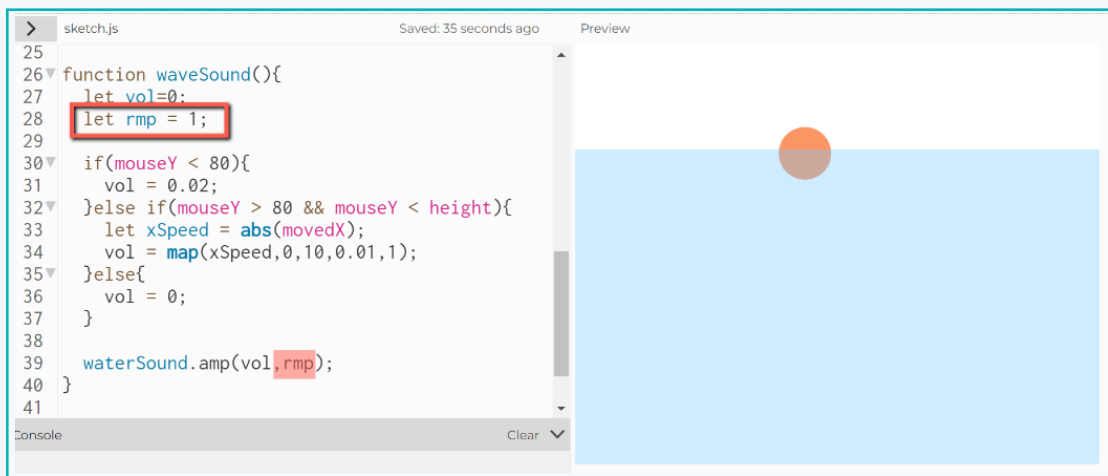
.amp(vol, [rampTime]): The 'rampTime' parameter creates a fade effect or down ramp that lasts for 'rampTime'. This refers to the duration required to slowly reduce the amplitude to '0'.

This feature provides the desired ramp effect in the wave sound.

Activity

Step 9: Reduce the Amplitude

- Create a local variable 'rmp' in the 'waveSound()' function at the top and assign it with a value of '1', which represents the time required to reduce the amplitude.
- Add the second parameter 'rmp' in the 'amp()' command by inserting a comma after 'vol'.



```
> sketch.js Saved: 35 seconds ago Preview
25
26 function waveSound(){
27   let vol=0;
28   let rmp = 1;
29
30   if(mouseY < 80){
31     vol = 0.02;
32   }else if(mouseY > 80 && mouseY < height){
33     let xSpeed = abs(movedX);
34     vol = map(xSpeed,0,10,0.01,1);
35   }else{
36     vol = 0;
37   }
38
39   waterSound.amp(vol, rmp);
40 }
41
Console Clear
```

Now, if we move the object in water, the sound of waves is generated in real-time.

Exercise

State if the statements are True or False

- 3** The 'amp()' command accepts only one input parameter.
▲ _____
- 4** We can set the amplitude of the sound as well as add the 'fade effect' using the 'amp()' command.
▲ _____

Exercise

 **State if the statements are True or False**

5 The 'abs()' command calculates the absolute value (magnitude) of a number.

 _____

6 We can use the commands related to sound even without linking it to the p5 sound library.

 _____

Activity

Whenever we go underwater, the sound of water changes. As discussed earlier, the human ear cannot perceive sound in water as it does in air. It can detect lower frequencies better in water.

As the object moves underwater, the sound should have lower frequencies or a lower pitch to accurately simulate underwater sounds. We can achieve this using the 'rate()' command.

.rate(): This command sets the playback rate of an audio file. It accepts the input value as a number where '1.0' is the normal speed, '0.5' is half the speed, etc. It changes the pitch of the sound. Lower the speed, lower is the pitch or the frequency of the sound.

We can achieve the underwater sound effect by mapping the y position of the object (mouseY) to the input for the 'rate()' command ranging from '0' to '1'. As we move downwards, the pitch of the sound decreases and vice-versa.

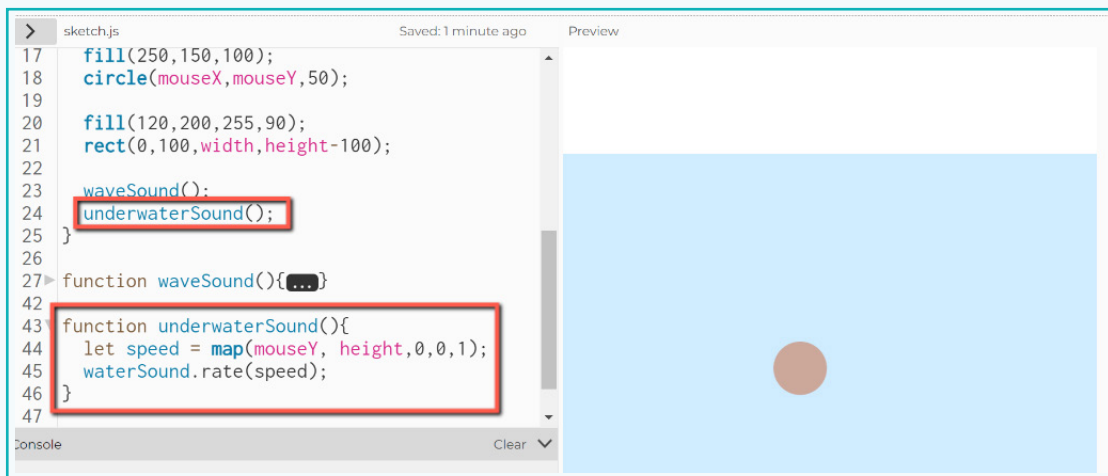
Step 10: Add Underwater Sound

1 Create a separate function 'underwaterSound()'.

2 Use the 'map' command to map the 'mouseY' value from range 'height - 0' to range '0 - 1' and assign it to a local variable 'speed'.

Activity

- 3 Add the 'waterSound.rate(speed)' command to change the pitch of the sound according to the object's position.
- 4 Call the 'underwaterSound()' function in draw().



```
17 fill(250,150,100);
18 circle(mouseX,mouseY,50);
19
20 fill(120,200,255,90);
21 rect(0,100,width,height-100);
22
23 waveSound();
24 underwaterSound();
25
26
27 function waveSound(){...}
28
29 function underwaterSound(){
30   let speed = map(mouseY, height,0,0,1);
31   waterSound.rate(speed);
32 }
33
```

We can now hear the underwater sound as the object moves deeper into the water.

This way, we have successfully completed the simulation of water sounds based on user movements and position using real-time sound synthesis.

The world is full of complex and wonderful sounds. In this lesson, we learnt how to replicate the sound of a virtual water pool in real-time using 'sound synthesis' with the help of 'p5js'. This way, it is now feasible to mix and digitally create more interesting and new sounds to accompany your videos and audio.

4. Sound Synthesis

Sound is synthesized in real-time based on player actions and surroundings in video games as well.

So, you can now use this technique to create realistic or artificial sounds and create a variety of audio, for videos and games.



Exercise

Answer the questions with one sentence.

7 What is the 'fade effect'?



8 Consider a variable 'inp' that represents the 'x' position of the cursor in the canvas, ranging from the left canvas edge to the right. How can we get an output value that will range from '360' to '0' degrees based on position of the cursor? The cursor at left edge position of the canvas should map to '360' degrees and right edge position of the canvas should map to '0' degrees. Write the command with proper inputs.



Tech Flash

- **Sound Synthesis** : It is the technique of generating sound digitally, using electronic hardware or software.
- **Real-Time Sound Synthesis** : It is a sound synthesis technique that uses real-time user input parameters to produce sound in real-time.
- **p5 Sound** : A JavaScript library that provides web audio functionalities like audio recording, playback, analysis, and synthesis.
- **Function** : It is a block of code that performs a specific task allocated to it. It can be used in any code without repeating all the steps within the code block.
- **abs()** : This Command calculates the absolute value or magnitude of a number.